MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana

Department of CSE(Artificial Intelligence and Machine Learning)

ALGORITHMIC PROBLEM-SOLVING HACKATHON (APSH 2026)

PROJECT DOCUMENTATION

Date: 03-10-2026 | Venue: 3rd Floor, A & B Blocks

Field	Details
Project Title	AMR Warehouse Robot Pathfinding Visualizer (Problem 61: Warehouse Robot Navigation)
Team Name / Team No.	05
Year / Semester / Section	II Year B.Tech / I Sem / F Section
Algorithm Paradigm Used	Graph algorithms – shortest-path search on a weighted grid graph: A* (heuristic best-first), Dijkstra's algorithm (greedy, uniform-cost) and Breadth-First Search (baseline)
Programming Language / Tools	Python 3 (NumPy, Matplotlib, Pygame-CE, Streamlit, pandas, Pillow); HTML5 / CSS3 / JavaScript (Canvas) frontend; Python http.server REST API
Faculty Mentor	Alekhya Ma'am & Shivmani Sir

Team Members

S.No	Name of the Student	Roll No.	Role
1	Manthan Mahadev Biradar	25MVCSMR0512	Team Lead
2	Yashvardhan Singh Gaur	25MVCSMR0540	Backend Development
3	Jampala Purnasena	25MVCSMR0491	Backend Development
4	Biswajit Behera	25MVCSMR0472	Frontend Development

1. Problem Statement

State the problem clearly, with its inputs, outputs and constraints.

Problem 61 – Warehouse Robot Navigation. An Autonomous Mobile Robot (AMR) in a warehouse must travel from its docking station to a pickup location while avoiding shelf racks and walls and, where possible, avoiding congested aisles. The task is to compute the cheapest collision-free route. The warehouse is modelled as a grid graph: every cell is a node and every pair of orthogonally adjacent walkable cells is joined by an edge.

Inputs

- A warehouse grid of R rows × C columns (default 20 × 25; the layout generators accept any size).
- Each cell has one of six types: Floor (cost 1), Heavy-Traffic zone (cost 3 by default, 5 in the traffic-depot preset), Obstacle/steel wall (impassable), Shelf rack (impassable), Robot Start dock and Pickup Goal (both cost 1).
- A start cell S and a goal cell G.

Output

- The ordered list of cells from S to G forming a minimum-cost path, with its total cost, its hop count (number of moves), the cells explored by the search and the execution time.
- If S or G is not walkable, or G is unreachable, the program reports that no path exists.

Constraints and assumptions

- Moves are 4-directional (up, down, left, right); diagonal moves are not allowed.
- The cost of a move is the cost of the cell being entered; all costs are non-negative (≥ 1 for walkable cells).
- The environment is static and a single robot is planned; the path must never enter an obstacle or shelf cell.
- The path returned must be optimal in total weighted cost, not just in number of moves.

2. Objectives

List 2–4 specific objectives of the project.

1. **Model** the warehouse as a weighted grid graph that supports obstacles, shelf racks and heavy-traffic zones with different traversal costs.
2. **Implement** three solvers behind one common interface – A* search (Manhattan heuristic), Dijkstra's algorithm and Breadth-First Search – and guarantee a minimum-cost path for the weighted solvers.
3. **Compare** the solvers experimentally on five warehouse layouts using path cost, hop count, cells explored and run time, and show why a heuristic reduces the search effort and why plain BFS is unsuitable when costs differ.
4. **Visualise** every step of each search (frontier, explored cells, g/f/h values, final path) through a full-stack web application, so the algorithms can be demonstrated and edited interactively.

3. Proposed Approach

Explain the algorithm design paradigm chosen and why it suits this problem. Mention alternatives considered.

Paradigm chosen: graph shortest-path search. The warehouse floor is a graph, and finding the cheapest route between two nodes is the single-source/single-target shortest-path problem. Dijkstra's algorithm is a greedy algorithm: it always settles the unvisited cell with the smallest known cost-from-start g(n), which is safe because no edge cost is negative. A* keeps the same greedy structure but orders the priority queue by $f(n) = g(n) + h(n)$, where h(n) is the Manhattan distance to the goal, so the search is pulled towards the pickup point instead of spreading in all directions.

Why A* suits this problem. The robot needs one route between two known cells, and the goal coordinates are available, so a goal-directed search is natural. The Manhattan heuristic is admissible (it never overestimates) because each move changes the row or column by exactly one and costs at least 1. It is also consistent: $h(u) \leq \text{cost}(u,v) + h(v)$ for every edge, since h changes by at most 1 per move. Consistency means a cell popped from the queue already has

its optimal cost, so it never needs to be re-opened (the implementation relies on this). Therefore A* returns a path with exactly the same cost as Dijkstra while exploring fewer cells.

Alternatives considered

- **Dijkstra's algorithm** – optimal but blind: it explores every cell cheaper than the goal. Kept as the reference solver.
- **Breadth-First Search** – optimal only in the number of moves. On a weighted floor it can choose a short but congested route (shown in Section 7: cost 108 versus the optimal 28). Kept as the naive baseline.
- **Depth-First Search / backtracking** – does not guarantee the shortest path and may enumerate exponentially many routes; rejected.
- **Bellman-Ford (O(VE)) and Floyd-Warshall (O(V³))** – handle negative edges or all-pairs queries that are not needed here; too slow.
- **Diagonal moves, Jump Point Search, D* Lite, multi-robot planning** – useful extensions but outside the scope of this single-robot, static, 4-connected problem (see Future Scope).

4. Algorithm / Pseudocode

Write the step-by-step algorithm or pseudocode. Add a flowchart if needed.

A. Graph definition

```
Node        : cell (r, c) with 0 <= r < R, 0 <= c < C
Walkable    : cell type is not OBSTACLE and not SHELF
Neighbours  : (r-1,c), (r+1,c), (r,c-1), (r,c+1) that are walkable
Edge cost   : cost(v) = traversal cost of the cell v being entered
Heuristic   : h(v) = |r_v - r_G| + |c_v - c_G|          (Manhattan)
```

B. A* search (as implemented in algorithms/astar.py)

```
ASTAR(grid, S, G):
  if S or G not walkable: return NO_PATH
  if S == G: return path [S], cost 0
  g[S] = 0;  parent[S] = None;  explored = {}
  PQ = min-heap ordered by (f, h);  push (h(S), h(S), S)
  while PQ is not empty:
      (f, h, u) = pop smallest from PQ
      if u in explored: continue              # stale heap entry
      add u to explored; record animation step
      if u == G: break
      for each walkable neighbour v of u:
          if v in explored: continue
          g' = g[u] + cost(v)
          if v not in g or g' < g[v]:
              g[v] = g';  parent[v] = u
              push (g' + h(v), h(v), v)
  if G not reached: return NO_PATH
  path = follow parent[] from G back to S, then reverse
  return path, g[G], len(path) - 1, explored
```

C. Dijkstra and BFS (differences only)

- **Dijkstra:** identical loop with h(n) = 0, so the heap is ordered by g(n) alone.

- **BFS:** a FIFO queue (collections.deque) replaces the heap; a cell is marked visited when first discovered and its parent is never updated, so the result minimises hops. The weighted cost of that route is summed afterwards.

D. Flowchart of the A* search

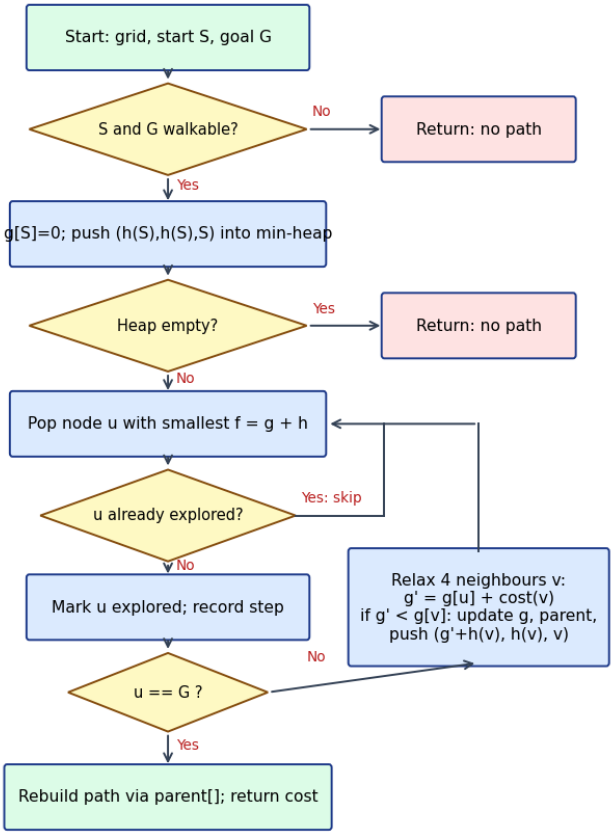


Figure 1: Flowchart of the A search implemented in the project.*

5. Complexity Analysis

Give the time and space complexity, with justification.

Let V = R × C be the number of cells and E the number of edges. Each cell has at most 4 neighbours, so E ≤ 4V = O(V). Let d be the number of moves on the optimal path and k the number of cells actually expanded (d ≤ k ≤ V).

The table below is for **A* search**, the proposed solver (binary heap from Python's heapq with lazy deletion):

Complexity	Best Case	Average Case	Worst Case
Time	O(d log d)	O(k log k), d ≤ k ≤ V	O(V log V)
Space	O(d)	O(k)	O(V)

Justification

- Each cell is expanded at most once (the explored set discards stale heap entries), so there are at most V expansions.
- Each edge is relaxed at most once per endpoint expansion, so there are O(E) = O(V) pushes; the heap never holds more than O(V) entries, and every push or pop costs O(log V). Total worst case: O(V log V) = O(R·C·log(R·C)).
- Best case: with a perfect heuristic A* expands only the cells on the path, giving O(d log d). Measured: on the empty arena and dense-rack layout A* expanded 44 cells for a 43-move path.
- Average case: depends on how well h guides the search. Measured on five layouts, A* expanded between 33 and 361 cells, versus 346 to 500 for Dijkstra.

- Space: the g-score map, parent map, explored set and heap each hold O(k) entries, so O(d) in the best case and O(V) in the worst case.

Comparison of the three solvers

Algorithm	Time (worst case)	Space (worst case)	Optimal on weighted grid?
A* (Manhattan)	O(V log V)	O(V)	Yes – heuristic is admissible and consistent
Dijkstra	O(V log V)	O(V)	Yes
BFS	O(V + E) = O(V)	O(V)	No – minimises hops, ignores cell costs

Note on the implementation. To drive the step-by-step animation, each solver stores a full snapshot (copies of the explored list and the g/f/h/parent maps) after every expansion. This makes the visualising version cost $O(k^2)$ time and memory for k expansions, although the underlying algorithm is O(V log V). It is irrelevant for the default 20 × 25 grid but became visible in testing: an 80 × 80 empty arena exhausted the memory of the test machine. Recording only the changes at each step would restore O(V) space (see Future Scope).

6. Implementation

Describe the main modules or functions, the data structures used and the tools or platform.

Architecture

The system has a Python pathfinding engine and a web frontend that communicate through a REST API. The solvers are written against a common abstract interface, so a new algorithm needs only a solve(grid, start, goal) method.

Module / file	Responsibility
warehouse/grid.py	WarehouseGrid class and CellType constants: NumPy grid and cost arrays, is_walkable(), get_cost(), get_neighbors() (4-directional), set_cell().
warehouse/presets.py	Five layout generators: Amazon Fulfillment, Bottleneck Maze, Dense Storage Racks, Heavy Traffic Depot, Empty Arena.
algorithms/base.py	BaseSolver abstract class, PathResult and AlgorithmStep dataclasses, reconstruct_path() using the parent map.
algorithms/astar.py	AStarSolver: heapq priority queue ordered by (f, h, node), Manhattan heuristic.
algorithms/dijkstra.py	DijkstraSolver: heapq priority queue ordered by g(n).
algorithms/bfs.py	BFSSolver: FIFO queue (deque), level map, hop-minimal path.
backend_server.py	http.server-based REST API (GET /api/status, GET /api/grid; POST /api/preset, /api/cell, /api/solve) that also serves the frontend on port 8000.
index.html, styles.css, app.js	Web frontend: three synchronised HTML5 canvases, play/step/scrub controls, paint tools, preset selector, benchmarking table and efficiency verdict.
app.py	Alternative Streamlit dashboard with Matplotlib charts.
standalone_pygame.py, visualization/	Desktop Pygame window; Pygame grid renderer; Matplotlib bar-chart and multi-path overlay generator.

Data structures used

- **NumPy 2-D arrays** for the cell-type grid and the cost grid (O(1) cell access).
- **Binary min-heap (heapq)** as the priority queue for A* and Dijkstra; **deque** as the FIFO queue for BFS.

- **Dictionaries** for g-scores, f-scores, h-scores, parent pointers and BFS levels; **sets** for the explored and frontier sets.
- **Dataclasses** (AlgorithmStep, PathResult) to carry per-step snapshots and final results; the backend serialises them to JSON.

Tools and platform

Python 3 (developed and run on 3.14) with NumPy, Matplotlib, Pygame-CE, Streamlit, pandas and Pillow (requirements.txt); the frontend uses plain HTML5, CSS3 and JavaScript with the Canvas API. Run with python backend_server.py and open http://localhost:8000.

7. Results and Test Cases

Give sample inputs and outputs and attach screenshots. Compare performance with a naive approach, if applicable.

All results below were produced by running the project's own solvers (the same 20 × 25 grids the web application loads). Execution time is the median of 30 runs on the machine used to prepare this report and will differ on other hardware; cost, hops and explored-cell counts are deterministic.

Test 1 – Five warehouse layouts (20 × 25 grid)

Layout	Algorithm	Path cost	Hops	Cells explored	Time (ms)
Amazon Fulfillment	A*	41.0	39	264	6.7
	Dijkstra	41.0	41	438	8.3
	BFS	41.0	39	436	7.9
Bottleneck Maze	A*	56.0	56	361	7.8
	Dijkstra	56.0	56	421	4.7
	BFS	56.0	56	421	5.8
Dense Storage Racks	A*	43.0	43	44	0.23
	Dijkstra	43.0	43	346	3.1
	BFS	43.0	43	346	3.5
Heavy Traffic Depot	A*	28.0	28	33	0.20
	Dijkstra	28.0	28	400	5.6
	BFS	**108.0**	24	358	7.9
Empty Arena	A*	43.0	43	44	0.32
	Dijkstra	43.0	43	500	7.0
	BFS	43.0	43	500	14.2

Observations. (1) All three solvers returned a path in every layout. (2) A* and Dijkstra always returned the same optimal cost. (3) A* explored 39.7% fewer cells than Dijkstra on the Amazon layout, 14.3% fewer on the Bottleneck Maze, 87.3% fewer on Dense Racks, 91.8% fewer on the Heavy Traffic Depot and 91.2% fewer on the Empty Arena. The saving is smallest in the maze, where walls force the search away from the straight line to the goal and the Manhattan heuristic is least informative. (4) In the Heavy Traffic Depot BFS found the path with the fewest moves (24) but its cost was 108, almost four times the optimal 28, because it ran through the cost-5 corridor. (5) In the

Bottleneck Maze A* was not faster in wall-clock time despite exploring fewer cells, because each A* step does more bookkeeping (f, g and h maps); with only ~400 cells this overhead is comparable to the saving.

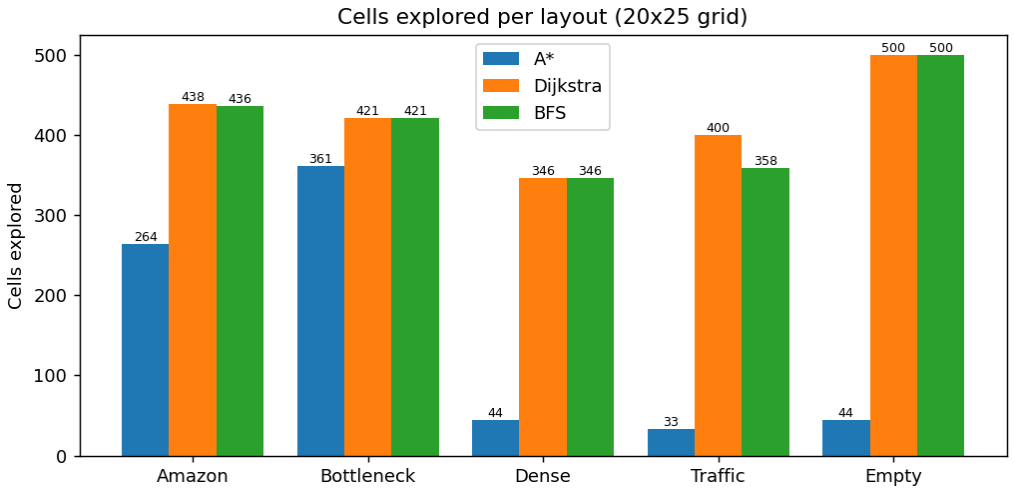


Figure 2: Cells explored by A, Dijkstra and BFS on each layout.*

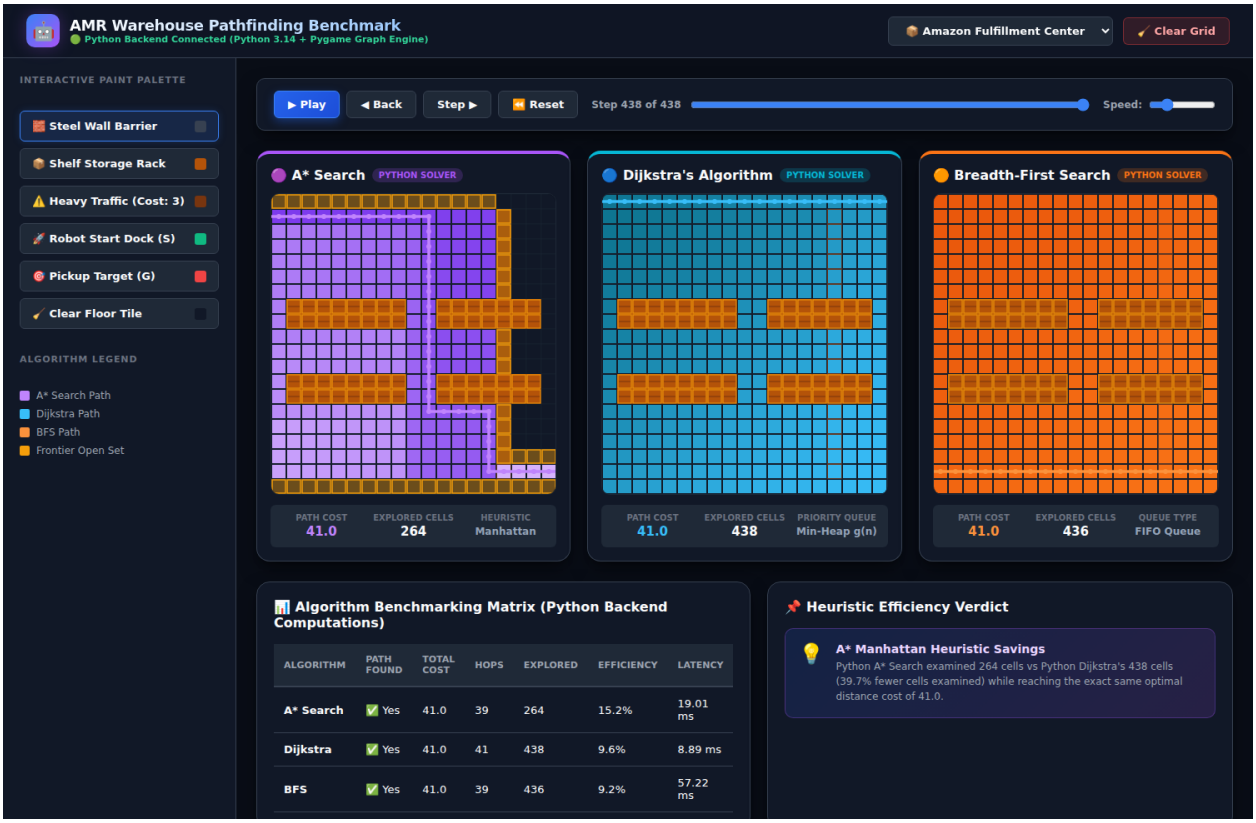


Figure 3: Web application (Amazon Fulfillment layout) after all three searches have finished, with the benchmarking table.

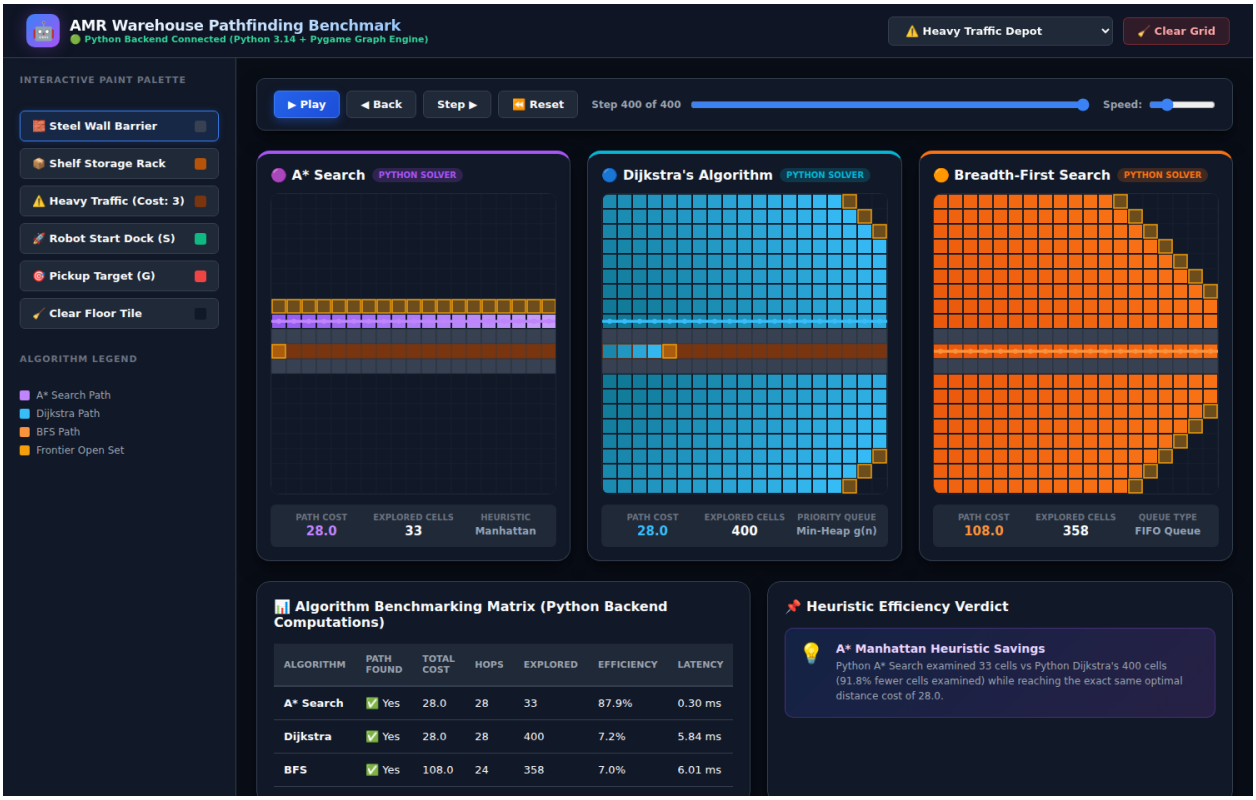


Figure 4: Heavy Traffic Depot – A (33 cells, cost 28) and Dijkstra (400 cells, cost 28) avoid the cost-5 corridor; BFS (358 cells) takes it and pays cost 108.*

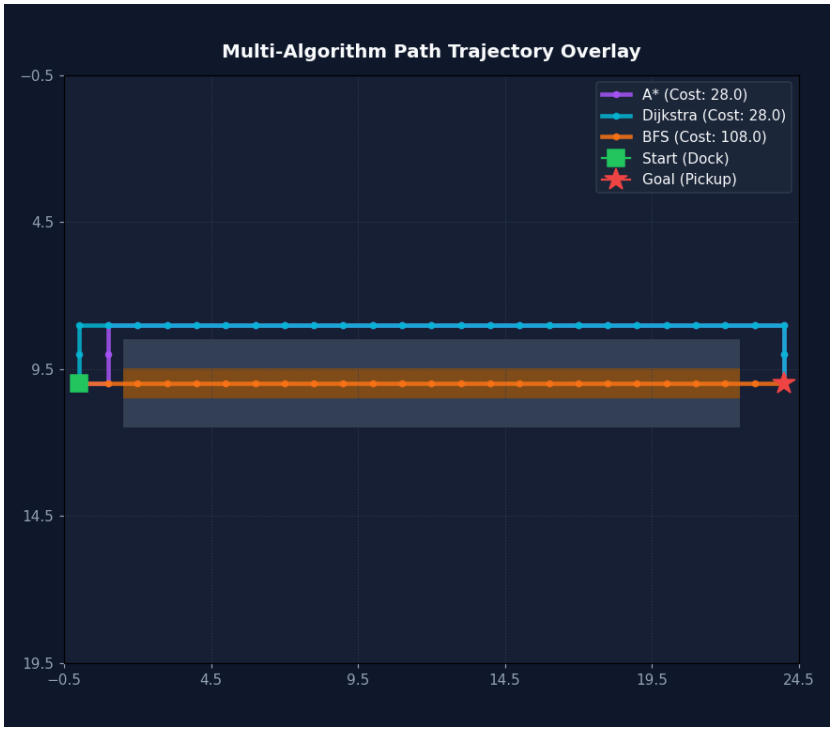


Figure 5: Path overlay generated by visualization/chart_generator.py for the Heavy Traffic Depot. A and Dijkstra share the same optimal route.*

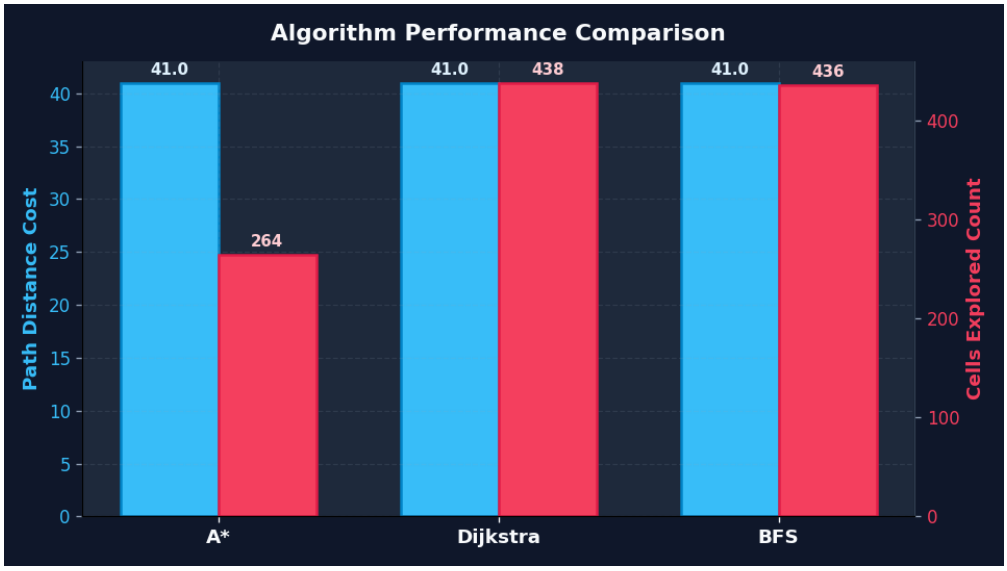


Figure 6: Path cost and cells explored for the Amazon Fulfillment layout (chart_generator.py).

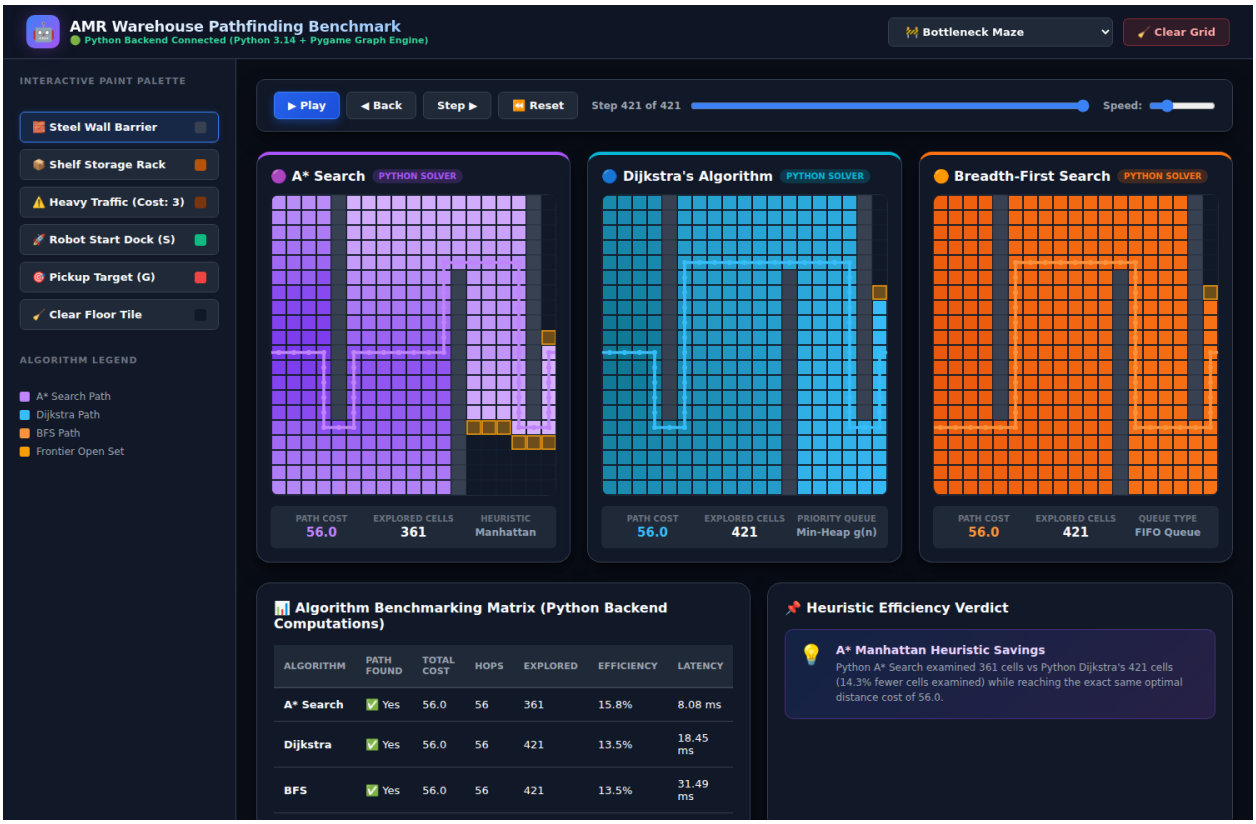


Figure 7: Bottleneck Maze – the walls limit the benefit of the heuristic (A: 361 cells, Dijkstra/BFS: 421).*

Test 2 – Edge cases

Case	Expected behaviour	Result (A*, Dijkstra, BFS)
Start equals goal	Zero-length path	Found; cost 0, hops 0 – all three
Wall splitting a 10 × 10 grid in two	Report no path after exhausting the reachable region	Not found; each solver explored all 50 reachable cells
Goal placed on an obstacle	Reject immediately	Not found; no cells explored

Test 3 – Randomised cross-check

500 random grids were generated (5–18 rows and columns, 25% obstacles, 10% heavy-traffic cells of cost 3, fixed random seed 7, start at the top-left and goal at the bottom-right). 336 grids were solvable and 164 were not. On every

grid, all three solvers agreed on whether a path exists; on all 336 solvable grids A* and Dijkstra returned exactly the same cost, and A* explored no more cells than Dijkstra. BFS returned a higher-than-optimal cost on 226 of the 336 solvable grids, confirming that it is not suitable once traffic costs are present.

Test 4 – Growth with grid size (empty n × n arena)

Grid	Cells (V)	A* explored	Dijkstra / BFS explored
10 × 10	100	19	100
20 × 20	400	39	400
40 × 40	1,600	79	1,600

A* expands only the 2n – 1 cells along the path (linear in n), whereas Dijkstra and BFS expand every cell (n^2). This matches the best-case analysis in Section 5.

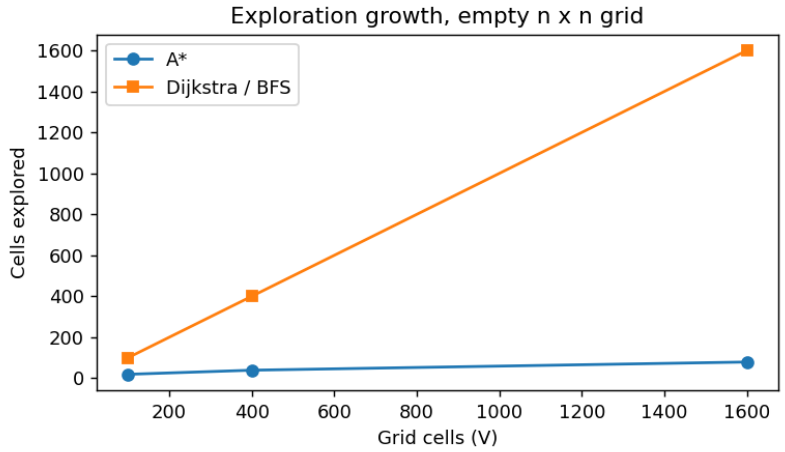


Figure 8: Cells explored versus grid size.

Comparison with the naive approach

BFS is the naive baseline. It matches the optimum only on unit-cost floors; with traffic zones it was up to 3.9× more expensive (108 vs 28). Dijkstra, the uninformed weighted baseline, is always optimal but explores 1.2× to 12× more cells than A* in the five layouts tested (438 vs 264, 421 vs 361, 346 vs 44, 400 vs 33, 500 vs 44).

8. Conclusion and Future Scope

Summarise the outcome and the improvements possible.

Conclusion

The project models warehouse navigation as a shortest-path problem on a weighted grid graph and implements A*, Dijkstra and BFS behind a common interface. On the tested layouts A* returned the optimal path in every case while expanding up to 91.8% fewer cells than Dijkstra, and BFS was shown to be unsuitable once congestion costs are present (cost 108 versus 28). A randomised check of 500 grids found no case where A* and Dijkstra disagreed. The full-stack visualiser (Python REST backend plus a Canvas frontend) makes each step of the searches visible and lets the user edit the warehouse and re-run the comparison.

Limitations

- Single robot, static environment and 4-directional movement; robot size, turning cost and acceleration are not modelled.
- Per-step snapshots make the visualising solvers $O(k^2)$ in memory (Section 5), limiting very large grids.
- The Manhattan heuristic is admissible only while every walkable cell costs at least 1.

- The backend keeps one global grid, so it supports a single user session.

Future scope

- Multi-robot planning with conflict-free routes (prioritised planning or Conflict-Based Search).
- Dynamic obstacles and re-planning with D* Lite or LPA*.
- 8-directional movement with an octile heuristic, turn penalties, Jump Point Search and bidirectional A*.
- Order-picking: visiting several pickup points in an efficient order (travelling-salesman style).
- Storing only the changes at each step to reduce visualisation memory to O(V), and adding automated unit tests.

9. References

List textbooks, research papers and online resources (IEEE format).

[1] P. E. Hart, N. J. Nilsson and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, Jul. 1968.

[2] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, no. 1, pp. 269–271, 1959.

[3] E. F. Moore, "The shortest path through a maze," in Proc. Int. Symp. on the Theory of Switching, Part II, Harvard Univ. Press, 1959, pp. 285–292.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[5] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[6] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357–362, 2020.

[7] J. D. Hunter, "Matplotlib: A 2D graphics environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.

[8] Python Software Foundation, "heapq – Heap queue algorithm," Python 3 Documentation. [Online]. Available: https://docs.python.org/3/library/heapq.html

[9] A. Patel, "Introduction to the A* algorithm," Red Blob Games. [Online]. Available: https://www.redblobgames.com/pathfinding/a-star/introduction.html